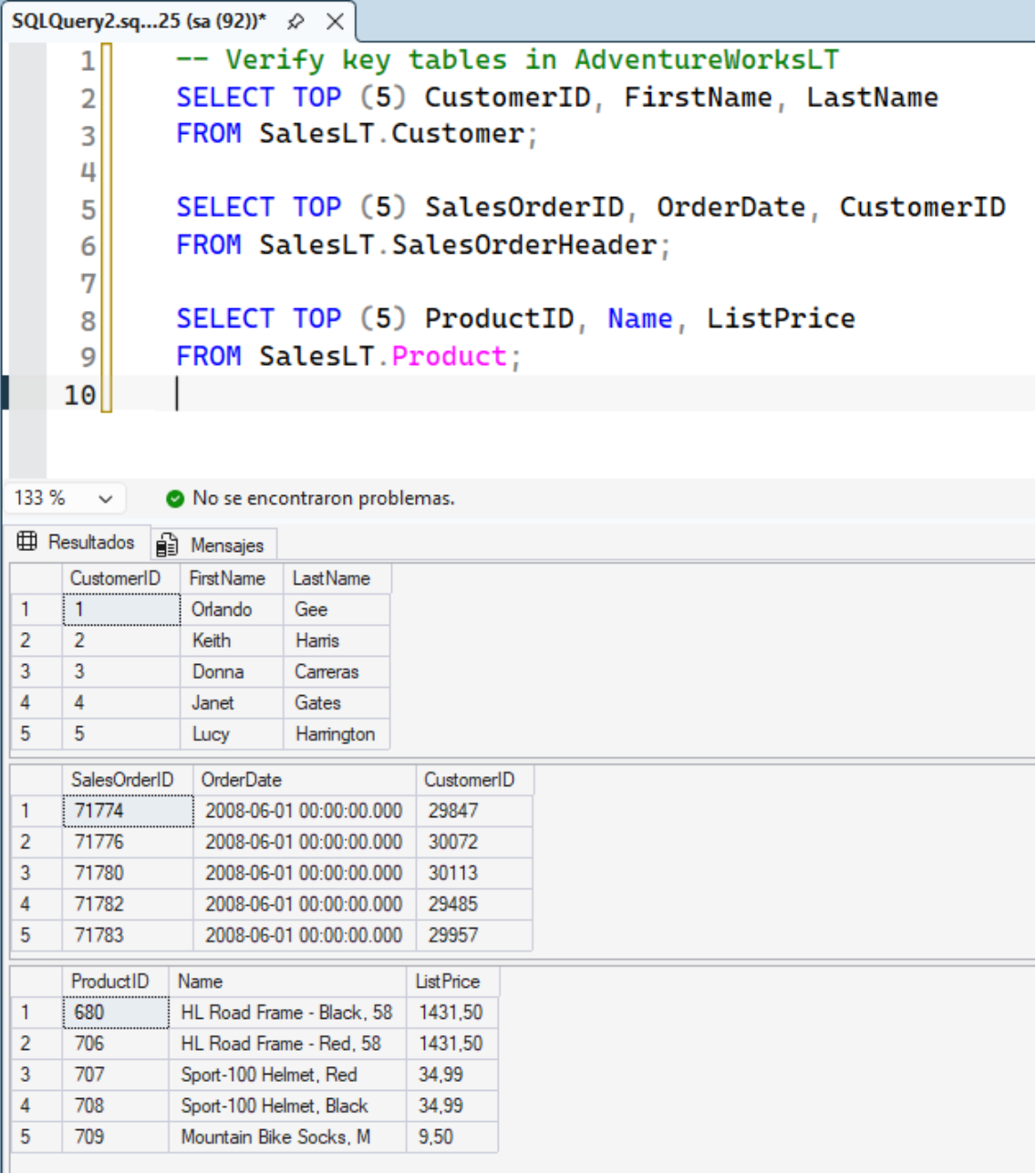


Implement programmability objects with SQL

Connect to AdventureWorksLT

Conexión a la base de datos AdventureWorksLT y verificación de las tablas clave: se consultan los 5 primeros registros de Customer, SalesOrderHeader y Product para comprobar el acceso a los datos de clientes, pedidos y productos.



The screenshot shows a SQL query window titled "SQLQuery2.sql...25 (sa (92))*". The query contains three SELECT statements to verify key tables in AdventureWorksLT. Below the query, the results are displayed in a grid format, showing the first 5 records for each table: Customer, SalesOrderHeader, and Product.

```
1  -- Verify key tables in AdventureWorksLT
2  SELECT TOP (5) CustomerID, FirstName, LastName
3  FROM SalesLT.Customer;
4
5  SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
6  FROM SalesLT.SalesOrderHeader;
7
8  SELECT TOP (5) ProductID, Name, ListPrice
9  FROM SalesLT.Product;
10
```

133 % No se encontraron problemas.

Resultados Mensajes

	CustomerID	FirstName	LastName
1	1	Orlando	Gee
2	2	Keith	Harris
3	3	Donna	Carreras
4	4	Janet	Gates
5	5	Lucy	Harrington

	SalesOrderID	OrderDate	CustomerID
1	71774	2008-06-01 00:00:00.000	29847
2	71776	2008-06-01 00:00:00.000	30072
3	71780	2008-06-01 00:00:00.000	30113
4	71782	2008-06-01 00:00:00.000	29485
5	71783	2008-06-01 00:00:00.000	29957

	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431,50
2	706	HL Road Frame - Red, 58	1431,50
3	707	Sport-100 Helmet, Red	34,99
4	708	Sport-100 Helmet, Black	34,99
5	709	Mountain Bike Socks, M	9,50

Create a view to simplify queries

Creación de la vista SalesLT.vCustomerOrders, que combina las tablas Customer y SalesOrderHeader mediante un INNER JOIN para mostrar el nombre completo

del cliente (CONCAT de FirstName y LastName) junto con el identificador y la fecha de cada pedido. La vista se creó correctamente.

SQLQuery2.sql...25 (sa (92))*

```
1 CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
2 SELECT
3     c.CustomerID,
4     CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
5     h.SalesOrderID,
6     h.OrderDate
7 FROM SalesLT.Customer c
8 INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```

133 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:09:26.6925846+02:00

Consulta de los 5 pedidos más recientes a través de la vista vCustomerOrders, ordenados por fecha descendente. El resultado muestra el CustomerID, el nombre del cliente, el SalesOrderID y la fecha del pedido.

SQLQuery2.sql...25 (sa (92))*

```
1 SELECT TOP (5) *
2 FROM SalesLT.vCustomerOrders
3 ORDER BY OrderDate DESC;
```

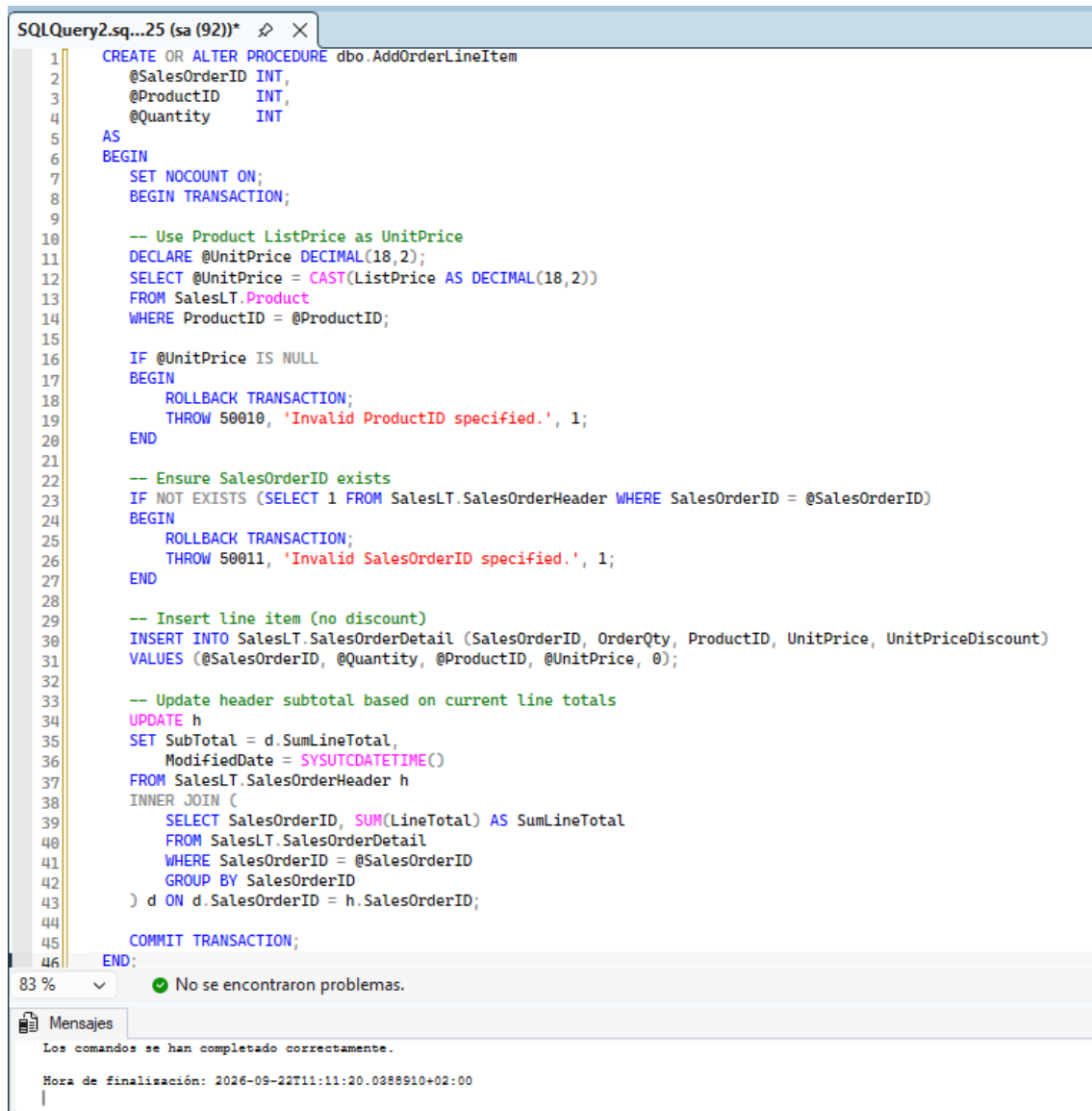
133 % 2 0

Resultados Mensajes

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

Create a stored procedure to process an order

Creación del procedimiento almacenado `dbo.AddOrderLineItem`, que recibe un `SalesOrderID`, un `ProductID` y una cantidad, obtiene el precio del producto, valida que el `ProductID` y el `SalesOrderID` existan (con `ROLLBACK` y `THROW` en caso de error), inserta la línea de pedido en `SalesOrderDetail` y actualiza el subtotal de la cabecera del pedido dentro de una transacción. Se creó correctamente.



```
SQLQuery2.sql...25 (sa (92))*
1 CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
2   @SalesOrderID INT,
3   @ProductID    INT,
4   @Quantity     INT
5 AS
6 BEGIN
7   SET NOCOUNT ON;
8   BEGIN TRANSACTION;
9
10  -- Use Product ListPrice as UnitPrice
11  DECLARE @UnitPrice DECIMAL(18,2);
12  SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
13  FROM SalesLT.Product
14  WHERE ProductID = @ProductID;
15
16  IF @UnitPrice IS NULL
17  BEGIN
18    ROLLBACK TRANSACTION;
19    THROW 50010, 'Invalid ProductID specified.', 1;
20  END
21
22  -- Ensure SalesOrderID exists
23  IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE SalesOrderID = @SalesOrderID)
24  BEGIN
25    ROLLBACK TRANSACTION;
26    THROW 50011, 'Invalid SalesOrderID specified.', 1;
27  END
28
29  -- Insert line item (no discount)
30  INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID, UnitPrice, UnitPriceDiscount)
31  VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);
32
33  -- Update header subtotal based on current line totals
34  UPDATE h
35  SET SubTotal = d.SumLineTotal,
36      ModifiedDate = SYSUTCDATETIME()
37  FROM SalesLT.SalesOrderHeader h
38  INNER JOIN (
39    SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
40    FROM SalesLT.SalesOrderDetail
41    WHERE SalesOrderID = @SalesOrderID
42    GROUP BY SalesOrderID
43  ) d ON d.SalesOrderID = h.SalesOrderID;
44
45  COMMIT TRANSACTION;
46 END;
```

83 %  No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:11:20.0388910+02:00

Ejecución del procedimiento `AddOrderLineItem` sobre un pedido existente, añadiendo una unidad del producto 680. La consulta posterior muestra la nueva línea insertada en `SalesOrderDetail` y el subtotal, impuestos, envío y total actualizados en `SalesOrderHeader`.

SQLQuery2.sql...25 (sa (92))*

```

1  -- Add a line item to an existing order (choose a valid SalesOrderID)
2  DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
3                                FROM SalesLT.SalesOrderHeader
4                                ORDER BY SalesOrderID DESC);
5  EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
6                             @ProductID = 680,
7                             @Quantity = 1; -- adjust ProductID as needed
8
9  SELECT TOP (5) *
10 FROM SalesLT.SalesOrderDetail
11 WHERE SalesOrderID = @SalesOrderID
12 ORDER BY SalesOrderDetailID DESC;
13
14 SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
15 FROM SalesLT.SalesOrderHeader
16 WHERE SalesOrderID = @SalesOrderID;

```

133 % 1 0

Resultados Mensajes

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal	rowguid	ModifiedDate
1	71946	113407	1	680	1431.50	0.00	1431.500000	3AC909B0-FCF0-4A93-8E48-75C5594F8F09	2026-09-22 11:13:03.820
2	71946	113406	1	916	31.584	0.00	31.584000	B571CEE0-A90A-4E76-9796-379B55352A28	2008-06-01 00:00:00.000

	SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
1	71946	1463.084	3.1163	0.9738	1467.1741

Create a scalar function for reusable calculations

Creación de la función escalar `dbo.fnOrderTotal`, que recibe un `OrderID` y devuelve la suma de `LineTotal` de sus líneas de detalle (con `ISNULL` para devolver 0 si no hay líneas). Se creó correctamente.

SQLQuery2.sql...25 (sa (92))*

```

1  CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
2  RETURNS DECIMAL(18,2)
3  AS
4  BEGIN
5      DECLARE @Total DECIMAL(18,2);
6
7      SELECT @Total = SUM(LineTotal)
8      FROM SalesLT.SalesOrderDetail
9      WHERE SalesOrderID = @OrderID;
10
11     RETURN ISNULL(@Total, 0.00);
12 END;

```

133 % No se encontraron problemas.

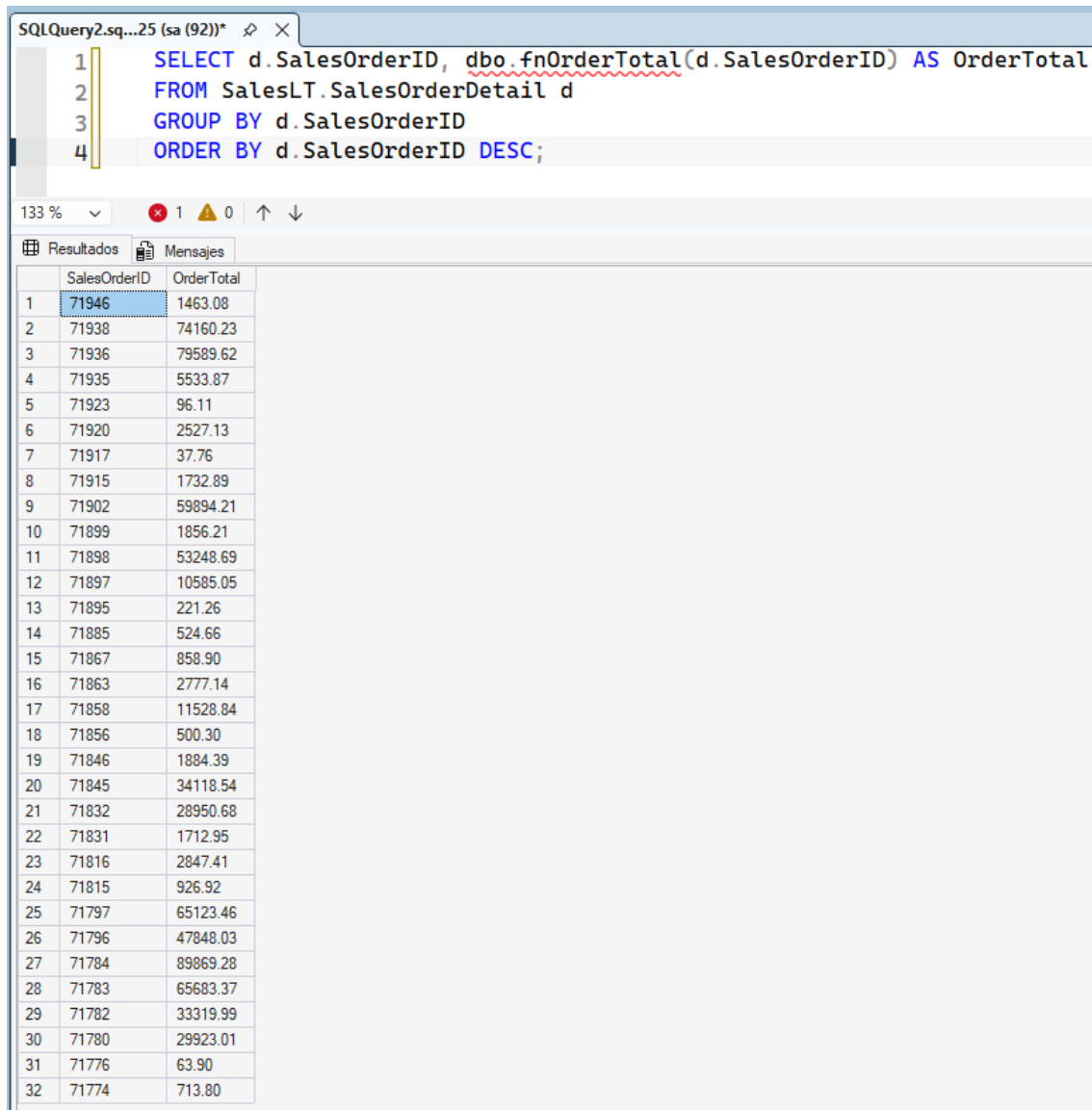
Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:13:55.6514566+02:00

Consulta que calcula el total de cada pedido mediante `fnOrderTotal`, agrupando por `SalesOrderID` y ordenando de mayor a menor. El resultado muestra el importe

total (OrderTotal) de cada uno de los pedidos.



The screenshot shows a SQL query window with the following text:

```
1 SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
2 FROM SalesLT.SalesOrderDetail d
3 GROUP BY d.SalesOrderID
4 ORDER BY d.SalesOrderID DESC;
```

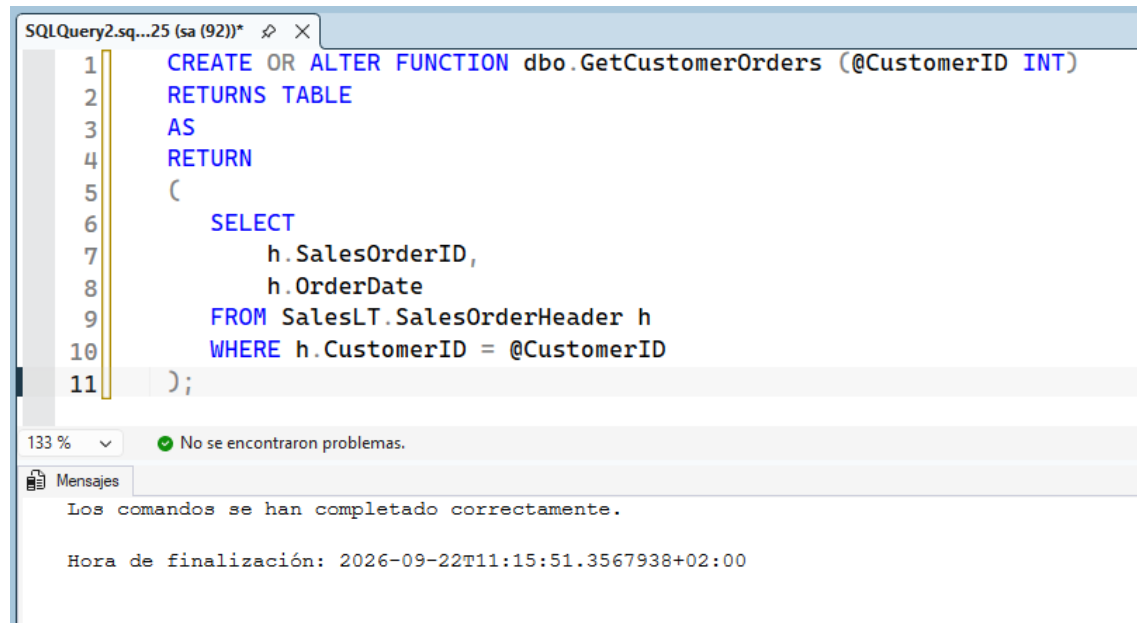
Below the query, the 'Resultados' (Results) tab is active, displaying a table with two columns: SalesOrderID and OrderTotal. The table contains 32 rows of data, sorted by SalesOrderID in descending order. The first row (SalesOrderID 71946) is highlighted in blue.

SalesOrderID	OrderTotal
71946	1463.08
71938	74160.23
71936	79589.62
71935	5533.87
71923	96.11
71920	2527.13
71917	37.76
71915	1732.89
71902	59894.21
71899	1856.21
71898	53248.69
71897	10585.05
71895	221.26
71885	524.66
71867	858.90
71863	2777.14
71858	11528.84
71856	500.30
71846	1884.39
71845	34118.54
71832	28950.68
71831	1712.95
71816	2847.41
71815	926.92
71797	65123.46
71796	47848.03
71784	89869.28
71783	65683.37
71782	33319.99
71780	29923.01
71776	63.90
71774	713.80

Create an inline table-valued function (TVF)

Creación de la función de tabla en línea `dbo.GetCustomerOrders`, que recibe un `CustomerID` y devuelve el `SalesOrderID` y la fecha de todos los pedidos de ese

cliente. Se creó correctamente.



```
1 CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
2 RETURNS TABLE
3 AS
4 RETURN
5 (
6     SELECT
7         h.SalesOrderID,
8         h.OrderDate
9     FROM SalesLT.SalesOrderHeader h
10    WHERE h.CustomerID = @CustomerID
11 ) ;
```

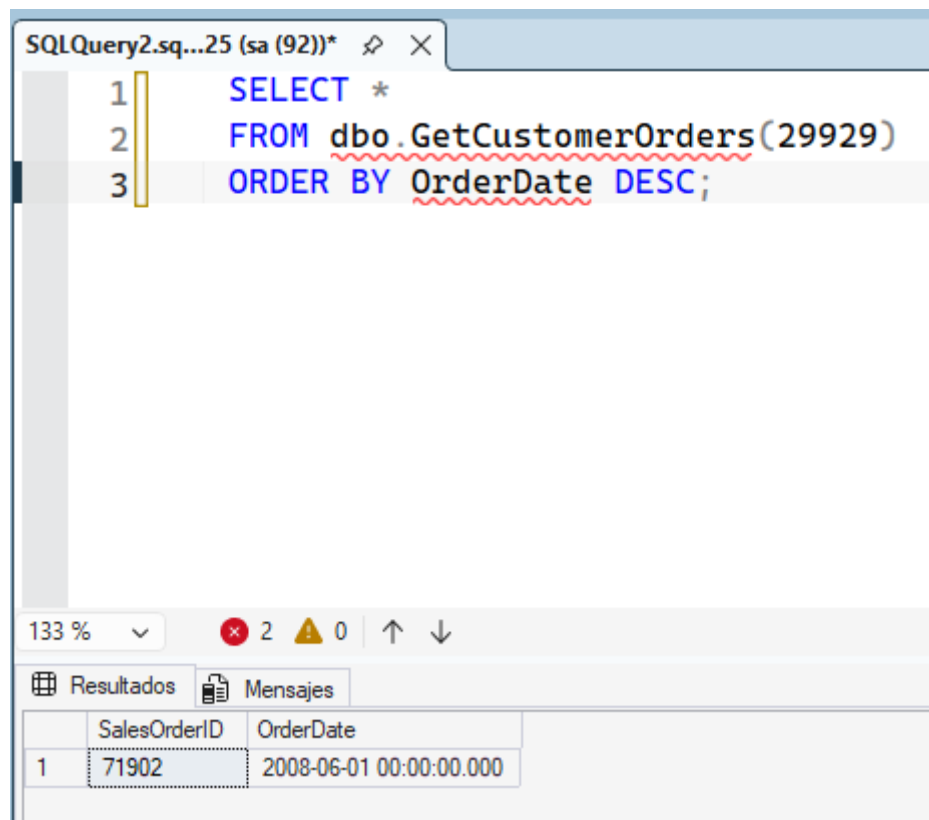
133 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:15:51.3567938+02:00

Consulta de los pedidos del cliente 29929 mediante la función GetCustomerOrders, ordenados por fecha descendente. El resultado muestra un pedido (71902) con su fecha correspondiente.



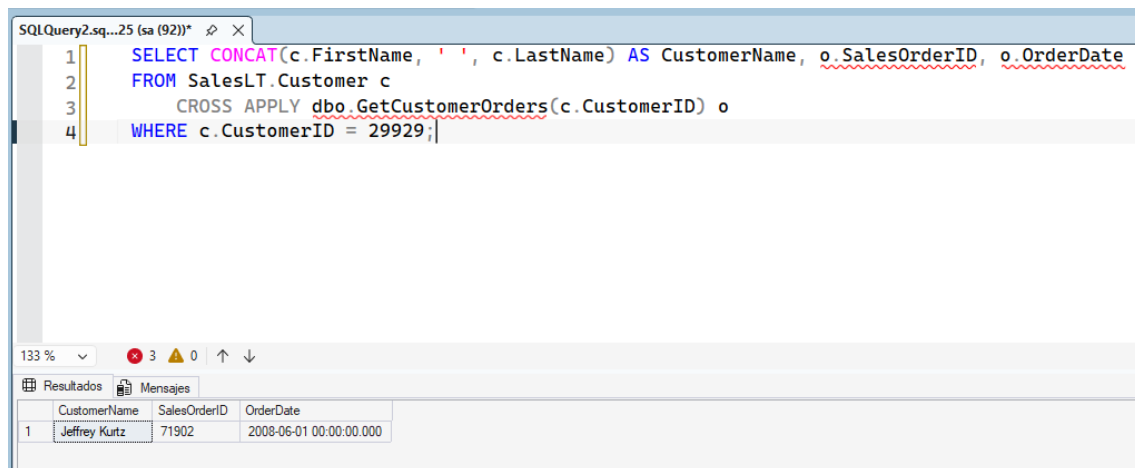
```
1 SELECT *
2 FROM dbo.GetCustomerOrders(29929)
3 ORDER BY OrderDate DESC;
```

133 % 2 0

Resultados Mensajes

	SalesOrderID	OrderDate
1	71902	2008-06-01 00:00:00.000

Consulta con CROSS APPLY que combina la tabla Customer con la función GetCustomerOrders para mostrar, en una sola fila, el nombre completo del cliente 29929 (Jeffrey Kurtz) junto con su pedido y fecha.



The screenshot shows a SQL query window with the following text:

```
1 SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
2 FROM SalesLT.Customer c
3 CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
4 WHERE c.CustomerID = 29929;
```

Below the query window, the 'Results' pane shows a table with 3 columns: CustomerName, SalesOrderID, and OrderDate. The table contains one row of data.

	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

Create a trigger to log changes

Creación de la tabla `dbo.OrderAudit` (si no existe) para registrar el historial de cambios en los totales de los pedidos, y del trigger

`SalesLT.trg_LogOrderTotalChange` sobre `SalesOrderDetail`, que se dispara tras `INSERT` o `UPDATE`. El trigger calcula el total anterior y el nuevo total de cada pedido afectado (combinando las filas `inserted` y `deleted`) y registra ambos valores

en OrderAudit. Se creó correctamente.

```
SQLQuery2.sql...25 (sa (92))*
1  -- Audit table
2  IF OBJECT_ID('dbo.OrderAudit') IS NULL
3  BEGIN
4      CREATE TABLE dbo.OrderAudit (
5          AuditID      INT IDENTITY(1,1) PRIMARY KEY,
6          OrderID      INT NOT NULL,
7          OldTotal     DECIMAL(18,2) NULL,
8          NewTotal     DECIMAL(18,2) NULL,
9          ChangedAt    DATETIME2 NOT NULL DEFAULT SYSUTCOATETIME()
10     );
11 END
12 GO
13
14 -- Trigger on order details updates
15 CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
16 ON SalesLT.SalesOrderDetail
17 AFTER INSERT, UPDATE
18 AS
19 BEGIN
20     SET NOCOUNT ON;
21
22     ;WITH AffectedOrders AS (
23         SELECT SalesOrderID FROM inserted
24         UNION
25         SELECT SalesOrderID FROM deleted
26     ),
27     -- New totals from the base table (already reflects changes)
28     NewTotals AS (
29         SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
30         FROM SalesLT.SalesOrderDetail d
31         INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
32         GROUP BY d.SalesOrderID
33     ),
34     -- Contribution of the newly inserted/updated rows
35     InsertedTotals AS (
36         SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
37         FROM inserted
38         GROUP BY SalesOrderID
39     ),
40     -- Contribution of the previous row versions (empty on INSERT)
41     DeletedTotals AS (
42         SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
43         FROM deleted
44         GROUP BY SalesOrderID
45     )
46     INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
47     SELECT
48         n.SalesOrderID,
49         n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
50         n.Total AS NewTotal
51     FROM NewTotals n
52     LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
53     LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
54 END;
```

75 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:18:12.2097395+02:00

Actualización de una línea de detalle (incremento de OrderQty) del pedido más reciente, lo que dispara el trigger. La consulta a OrderAudit muestra el registro

generado, con el OrderID, el total anterior (OldTotal), el nuevo total (NewTotal) y la fecha del cambio.

SQLQuery2.sql...25 (sa (92))

```
1  -- Update an order detail to change the total
2  UPDATE d
3  SET OrderQty = OrderQty + 1
4  FROM SalesLT.SalesOrderDetail d
5  WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader ORDER BY SalesOrderID DESC);
6
7  SELECT TOP (5) *
8  FROM dbo.OrderAudit
9  ORDER BY AuditID DESC;
```

133 % 2 0 133 % Línea: 9, Carácter: 24

Resultados Mensajes

	AuditID	OrderID	OldTotal	NewTotal	ChangedAt
1	1	71946	1463.08	2926.17	2026-09-22 09:19:03.7406321